Análise de Resiliência e Propagação de Vulnerabilidades em Grafos de Dependências do Ecossistema Python

Márcio Levi Sales Prado — RA 183680
MC859 — UNICAMP, Abril de 2026

1. Introdução

O ecossistema de pacotes Python (PyPI) é um dos maiores repositórios de software livre do mundo, com mais de 500 mil pacotes publicados. A cadeia de dependências entre esses pacotes forma uma rede complexa: quando um pacote amplamente usado contém uma vulnerabilidade de segurança, todos os projetos que dependem dele — direta ou indiretamente — ficam expostos ao mesmo risco.

Este projeto modela esse ecossistema como um **grafo direcionado**, no qual cada vértice representa um pacote e cada aresta **A** → **B** indica que A declara dependência direta de runtime sobre B. O objetivo central é identificar pacotes de alto risco combinando métricas estruturais do grafo (centralidade, grau, componentes fortemente conexas) com dados reais de uso (downloads mensais) e vulnerabilidades conhecidas do OSV (*Open Source Vulnerabilities*) e do GitHub Advisory Database.

Coleta de dados. Foram combinadas duas fontes: (i) os **8.000 pacotes mais baixados** nos últimos 30 dias (*top-pypi-packages*. hugovk.github.io) e (ii) **40.000 pacotes adicionais** do *PyPI Simple Index*. Para cada semente. a API JSON do PyPI extraiu dependências de runtime de requires_dist. ignorando extras opcionais. Com 20 threads paralelas e BFS transitivo. o grafo resultante tem **54.406 pacotes** e **178.193 arestas**. Cada nó foi anotado com downloads mensais para ponderar a propagação de vulnerabilidades.

Repositório com instâncias e scripts: https://github.com/marcinholsp/Projeto_MC859

2. Tamanho do Grafo

Métrica	Valor
Número de vértices (pacotes)	54.406
Número de arestas (dependências)	178.193
Grau médio (in + out)	6.55
In-degree médio (dependentes diretos)	3.28
Out-degree médio (dependências diretas)	3.28
In-degree máximo	6.479 (numpy)
Out-degree máximo	1000
CFCs totais	54.298
Maior CFC	11 vértices
CFCs singleton (1 vértice)	54.238
Componentes fracamente conexas (CFrCs)	14.003
Maior componente fracamente conexa	40.105 vértices

A densidade do grafo é baixa (178.193 arestas para 54.406 nós). o que é esperado em redes de dependências reais: cada pacote declara. em média. cerca de 3.3 dependências diretas. O in-degree máximo de 6.479 (numpy) evidencia a presença de *hubs* — pacotes que servem de base para milhares de outros — característica típica de redes livres de escala (*scale-free*).

3. Distribuição de Graus

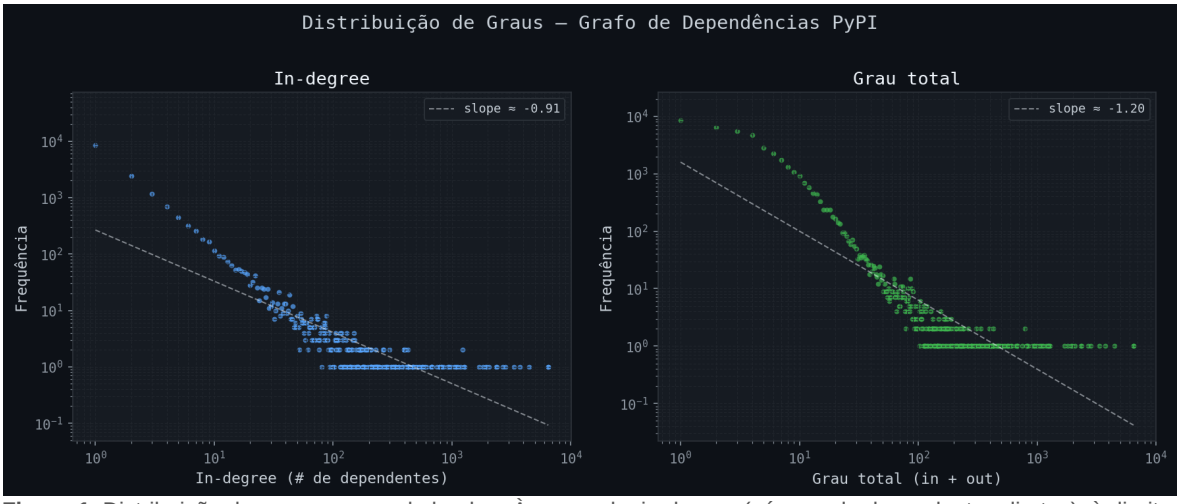

Figura 1. Distribuição de graus em escala log-log. À esquerda: in-degree (número de dependentes diretos); à direita: grau total (in + out). A linha tracejada mostra o ajuste por lei de potência.

A distribuição de graus segue uma **lei de potência** (*power-law*) em escala log-log, com expoentes estimados por regressão linear de –0,91 para o in-degree e –1,20 para o grau total. Esse padrão — característico de redes livres de escala — implica que a maioria dos pacotes tem poucos dependentes, enquanto um número muito reduzido de hubs concentra milhares deles. Do ponto de vista de segurança, esses hubs são candidatos naturais a pontos críticos: uma vulnerabilidade em numpy (in-degree 6.479), requests (6.463) ou pydantic (4.461) pode afetar, direta ou indiretamente, grande parte do ecossistema.

A Figura 2 apresenta os 20 pacotes com maior in-degree. numpy e requests destacam-se em uma faixa muito acima dos demais, seguidos por pydantic, pandas e pyyaml. Curiosamente, typing-extensions, líder em estudos restritos ao top-3.000 pacotes, aparece apenas em sexta posição quando o escopo é ampliado: muitos dos pacotes mais populares já dependem dele, mas ele tem alcance estrutural menor que bibliotecas científicas como numpy, que penetra em ferramentas de ML, dados e análise — ilustrando como o tamanho da amostra afeta o ranking de centralidade.

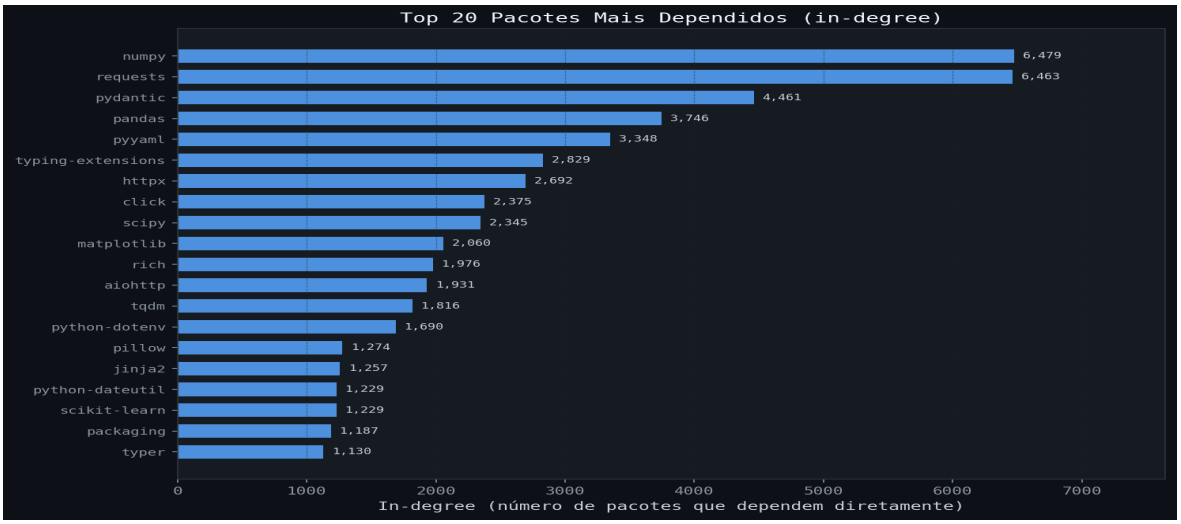

Figura 2. Top 20 pacotes por in-degree (número de pacotes que dependem diretamente de cada um).

4. Componentes Fortemente Conexas (CFCs)

O algoritmo de Tarjan identificou **54.298 CFCs**. Desse total. **54.238 são singletons** (99.89%). confirmando que o grafo de dependências PyPI é essencialmente *acíclico*. Gerenciadores de pacotes como pip exigem um grafo acíclico para resolver dependências deterministicamente. de modo que dependências circulares são proibidas em teoria — as raras exceções encontradas (60 CFCs com mais de um vértice. de tamanhos entre 2 e **11**) podem indicar versões de compatibilidade cruzada ou dependências opcionais não declaradas corretamente.

A Figura 3 ilustra a distribuição de tamanhos das CFCs. O painel esquerdo evidencia o domínio absoluto dos singletons; o painel direito amplia as CFCs com mais de um vértice, confirmando que dependências circulares são eventos raros e de pequena escala neste ecossistema.

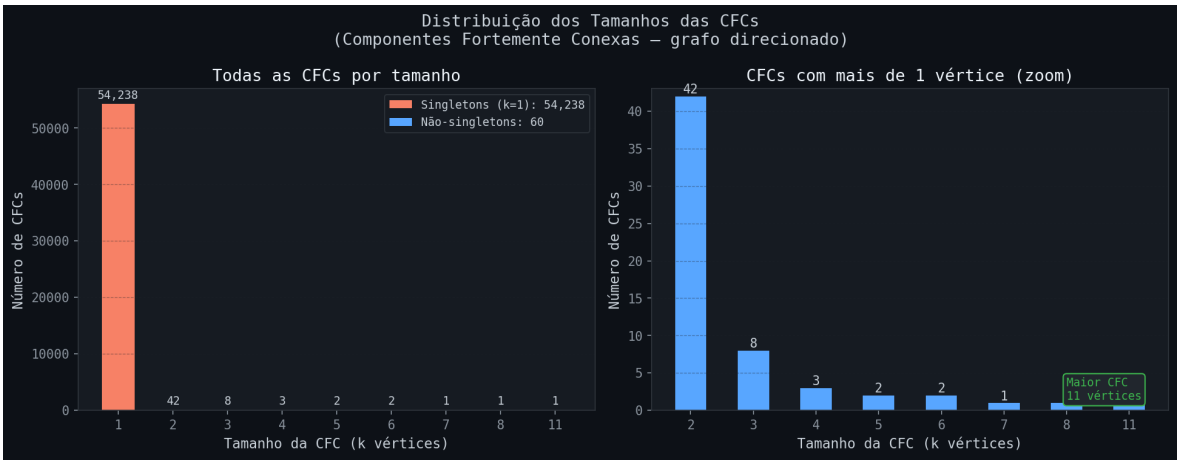

Figura 3. Distribuição dos tamanhos das CFCs. Esquerda: visão geral (54.238 singletons dominam). Direita: zoom nas 60 CFCs com mais de 1 vértice (tamanhos entre 2 e 11).

5. Componentes Fracamente Conexas (CFrCs)

Ignorando a direção das arestas. o grafo possui **14.003 componentes fracamente conexas**. A maior delas contém **40.105 vértices** (73.7% do total) — o chamado *componente gigante*. presente em praticamente todas as redes reais de larga escala. As demais 14.002 componentes são. em sua maioria. pacotes isolados ou pequenos subgrafos sem ligação com o núcleo principal do ecossistema — pacotes de nicho. abandonados. ou cujos metadados requires_dist não declaram dependências runtime.

A existência de um componente gigante cobrindo 73.7% do grafo tem implicações diretas para a análise de propagação de vulnerabilidades: uma falha em qualquer pacote dentro desse componente pode. em princípio. atingir até 40.104 outros pacotes por caminhos de dependência. Essa conectividade motivou o uso do modelo Independent Cascade para quantificar o alcance real de cada vulnerabilidade.

6. Análise de Vulnerabilidades

A base OSV (*Open Source Vulnerabilities*) e o GitHub Advisory Database foram consultados para cada nó do grafo. Foram identificados **648 pacotes com ao menos um CVE** (1.2% dos 54.406 pacotes totais). Para cada pacote vulnerável. o CVSS máximo foi extraído via biblioteca cvss e armazenado como atributo do nó no grafo anotado (pypi_dependency_graph_vuln.graphml).

Modelo Independent Cascade (IC) ponderado por downloads. Em vez de BFS/DFS determinístico, o projeto usa o modelo IC para responder: *se o pacote B for comprometido, com que probabilidade o pacote A que depende de B será afetado?* A probabilidade de propagação da aresta B → A (no grafo reverso) é proporcional à fração de downloads de B em relação a todas as dependências de A:

```
p(B→A) = log1p(downloads_B) / Σ log1p(downloads_dep), dep ∈ deps(A)
```

O score de criticidade combina gravidade real e alcance prático:

```
risk(v) = CVSS(v) × log1p(downloads_IC_afetados) × log1p(downloads_próprios)
```

500 simulações Monte Carlo foram executadas por pacote vulnerável. Esse score diferencia centralidade estrutural de impacto prático: um pacote com muitos dependentes mas pouco uso real obtém score baixo.

Pacote	CVSS	CVEs	Downloads próprios	Downloads IC afetados	Risk score
urllib3	9.8	28	1.42 B	2.73 B	4488
numpy	9.8	16	872 M	2.31 B	4350
pyyaml	9.8	8	847 M	655 M	4090
cryptography	9.1	28	958 M	1.73 B	4003
pygments	9.5	9	745 M	729 M	3960

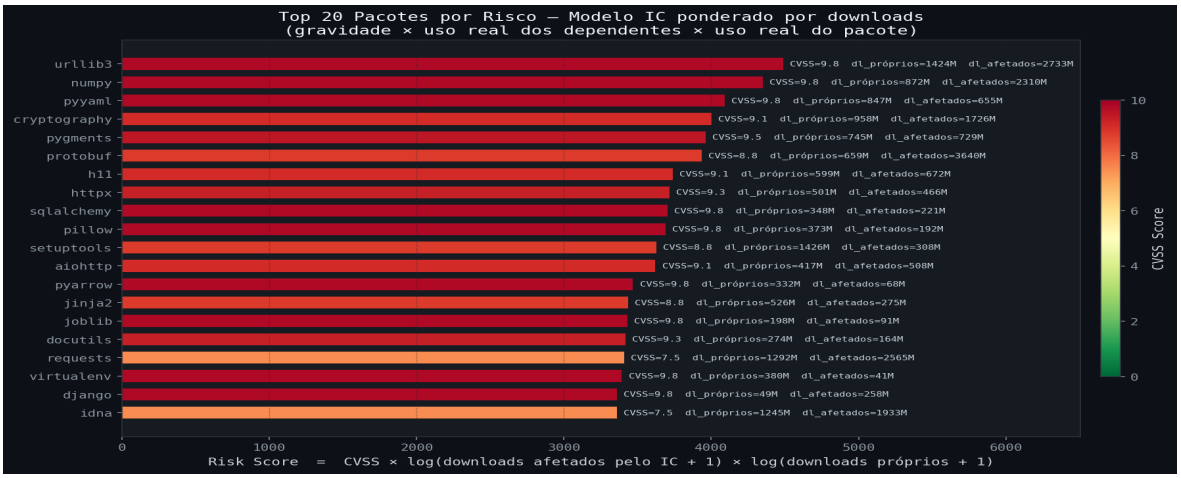

Figura 4. Top 20 pacotes por risco IC ponderado por downloads. A barra combina CVSS, alcance IC e volume de uso.

A Figura 5 (repositório) compara o alcance BFS estrutural com o alcance IC real: pacotes com muitos dependentes transitivos mas baixo IC são dependências opcionais ou pouco utilizadas na prática, evidenciando a importância de ponderar pelo uso real.

Instâncias disponíveis em: https://github.com/marcinholsp/Projeto_MC859 — formatos GraphML e GEXF, pasta data/.